

TUTORIAL 4

The goal of this week's tutorial would be to brush up on the concepts covered till now such as some of the fundamental questions related to operators, data types, and other constructs. The students MUST be comfortable with input() operations as well as **Typecasting**, by now.

Introduction to the basic way to take the elements of a list as input through the iterative approach[by **iteration**, '**LOOPS**' are being referred]

Method 1:

```
## What the students have used till now

list_of_elements = [] ## created an empty list
n = int(input("Enter the length of the list: "))
## taking n --> the number of elements in the list

## Let's run the loop
print("Enter list elements in successive lines: ")

for i in range(1, n+1):
    element = int(input()) ## element has been taken as input
    ## let's append it to the list
    list_of_elements.append(element)
##for loop ended

## Let's print the elements
print("")
print("The required list is:", list_of_elements)
```

Output

```
Enter the length of the list: 5
Enter list elements in successive lines:
34
21
44
1
4

The required list is: [34, 21, 44, 1, 4]
```

if there was an alternative to the above method? 'List comprehension'.

Method 2: [Uses split()]

```
## List comprehension
print("Enter space separated elements in single line:")
list_of_elements1 = [int(x) for x in input().split()]

print("the required list is: ", list_of_elements1)
```

Output:

```
Enter space separated elements in single line:
34 21 44 1 4
the required list is: [34, 21, 44, 1, 4]
```

Point to be explained to the students: why this method looks better than the previous approach?

- It is **cleaner**
- It saved our time. By not asking for a new element in each iteration. Instead, it asked for all the elements as input in a single separate line.
- This method performs **typecasting, splitting the string**[why? Because by default, the input element is a string in python] and adding(or appending) to the list in a **single line of code!!!**

A quick recap of function would be done at this point→

Now let's perform an operation to return the sum of elements of a list using a function called **listSummer()**.

This function accepts a list as a parameter and returns the sum of its elements.

Method 3:

```
def listSummer(another_list):
    ##Let's define the function
    sum_of_elements = 0
    for element in another_list:
        sum_of_elements += element

    return sum_of_elements

print("Enter space separated elements in single line:")
new_list = [int(x) for x in input().split()]

print("the required list is: ", new_list)
## let's pass this list to listSummer()

sum_value = listSummer(new_list) ##as the listSummer returns a value, it
```

```

                                ## has been assigned to the variable
sum_value

print("")
print("The sum of elements of the list is: ", sum_value)

```

Output:

```

Enter space separated elements in single line:
34 21 44 1 4
the required list is: [34, 21, 44, 1, 4]

The sum of elements of the list is: 104

```

Could it have been done easily and with few lines of code???

Yes!! Using the in-built **sum()** !!

Method 4:

```

print("Enter space separated elements in single line:")
list2 = [int(x) for x in input().split()]

print("the required list is: ", list2)

print("")
print(f"The sum of the list is: {sum(list2)}") ## wow!! f-string again!!!

```

```

Enter space separated elements in single line:
34 21 44 1 4
the required list is: [34, 21, 44, 1, 4]

The sum of the list is: 104

```

This example would reintroduce them to 'f-string' and in-built **sum()**. And it will conclude with these concepts:-

Typecasting, iterative approach, Lists, List Comprehension, split(), functions, assigning and using the returned value of a function, f-string reintroduction, sum() method.

But students must be aware about the benefit of doing certain things from scratch first(as we did for entering list elements in iteration and finding sum using user-defined function). This way they would really know what goes behind the easy and friendly functions such as **sum()** or **split()**!!